

High Performance Computing

Combined Lab Report: Assignment 06 & Assignment 07

Parallel Interpolation and Particle Mover using OpenMP

Dhirubhai Ambani University
Instructor: Dr. Bhaskar Chaudhury

April 2026

Contents

1	Introduction	3
2	Assignment 06 – Parallel Scatter Interpolation	4
2.1	Problem Statement	4
2.2	Implementation Approach	4
2.2.1	Data Structures	4
2.2.2	Parallelisation Strategy – Thread-Private Mesh + Critical Reduction	4
2.2.3	Pseudocode	5
2.2.4	Flowchart	7
2.2.5	Why This Approach	8
2.3	Results – Lab Machine	8
2.4	Results – Cluster	9
2.5	Analysis and Discussion	10
2.5.1	Race Condition Handling	10
2.5.2	Speedup Observations	10
2.5.3	Efficiency	11
2.5.4	Lab vs. Cluster	11
3	Assignment 07 – Parallel Interpolation with Particle Mover	12
3.1	Problem Statement	12
3.2	Implementation Approach	12
3.2.1	Data Structure Change	12
3.2.2	Stage 1: Interpolation	12
3.2.3	Stage 2: Normalisation	13
3.2.4	Stage 3: Mover	14
3.2.5	Stage 4: Denormalisation	14
3.2.6	Complete Pipeline Pseudocode	14
3.2.7	Flowchart	16
3.3	Results – Lab Machine	17

3.4	Results – Cluster	18
3.5	Phase Analysis	20
3.6	Analysis and Discussion	20
3.6.1	Speedup and Scalability	20
3.6.2	Load Imbalance	21
3.6.3	Lab vs. Cluster	21
3.6.4	Memory Access Patterns	21
3.6.5	Bottleneck Identification	21
4	Answers to Assignment Questions	22
4.1	Pseudocode and Diagrams	22
4.2	Race Condition Handling	22
4.3	Speedup and Execution Time Graphs	22
4.4	Parallel Efficiency	22
4.5	Maximum Speedup	22
4.6	Explanation of Speedup Behaviour	22
4.7	If Given Unlimited HPC Resources	23
4.8	Load Imbalance and Bottlenecks	23
4.9	Alternative Approaches	23
4.10	Interpolation vs. Mover Phase Bottleneck	24
5	Comparison: Assignment 06 vs. Assignment 07	25
6	Conclusion	25

1 Introduction

This report covers two consecutive lab assignments on parallel programming using OpenMP in C++. Both assignments deal with bilinear (linear) interpolation of scattered 2D particles onto a structured mesh. Assignment 06 focuses on parallelising the scatter (particle $\rightarrow$ mesh) interpolation step. Assignment 07 extends the work by adding a full particle-mover pipeline: scatter interpolation, grid normalisation, gather interpolation (mover), and grid denormalisation.

The domain is a unit square $[0, 1] \times [0, 1]$. A structured mesh of size $(N_x + 1) \times (N_y + 1)$ grid points covers the domain with uniform spacings $\Delta x = 1/N_x$ and $\Delta y = 1/N_y$. Each particle at position (x_i, y_i) contributes to the four surrounding grid nodes with bilinear weights.

Five benchmark configurations were used throughout both assignments:

Label	Config	N_x	N_y	Particles	Maxiter
A	Small grid, few particles	250	100	0.9 M	10
B	Small grid, many particles	250	100	5.0 M	10
C	Medium grid, many particles	500	200	3.6 M	10
D	Medium grid, very many	500	200	20.0 M	10
E	Large grid, many particles	1000	400	14.0 M	10

All benchmarks were run on two platforms:

- **Lab machine** – local workstation used for quick measurements.
- **Cluster** – dedicated HPC cluster nodes used for production timings. Results on the cluster are more reliable because the machine was exclusively reserved.

Thread counts tested: 1, 2, 4, 8, 16. The single-thread run was compiled *without* `-fopenmp` so it carries zero OpenMP overhead, giving a clean serial baseline.

2 Assignment 06 – Parallel Scatter Interpolation

2.1 Problem Statement

Given N particles at positions (x_i, y_i) with function values $f_i = 1$, accumulate their contributions onto the structured mesh using bilinear weights:

$$w_{00} = (\Delta x - l_x)(\Delta y - l_y) \quad (1)$$

$$w_{10} = l_x (\Delta y - l_y) \quad (2)$$

$$w_{01} = (\Delta x - l_x) l_y \quad (3)$$

$$w_{11} = l_x l_y \quad (4)$$

where $l_x = x_i - X_{\lfloor x_i/\Delta x \rfloor}$ and $l_y = y_i - Y_{\lfloor y_i/\Delta y \rfloor}$.

The difficulty in parallelising this step is that multiple particles may fall in the same or neighbouring grid cells, causing multiple threads to write to the same mesh location simultaneously – a classic race condition.

2.2 Implementation Approach

2.2.1 Data Structures

Two data structures are used:

- **Points** – a struct-of-arrays holding **x** and **y** coordinates for all N particles.
- **mesh_value** – a flat 1D array of size $(N_x + 1) \times (N_y + 1)$ representing the grid, accessed as `mesh[j * GRID_X + i]`.

Particles are read from a binary file in a single bulk **fread** call, avoiding per-particle I/O overhead that would dominate timing.

2.2.2 Parallelisation Strategy – Thread-Private Mesh + Critical Reduction

The chosen strategy is *privatisation*: each thread allocates its own private copy of the mesh, accumulates contributions independently without any synchronisation, and then a single critical section merges all private meshes into the shared one at the end.

```
1 void interpolation(double *mesh_value, Points *points) {
2     int grid_size = GRID_X * GRID_Y;
3
4     #pragma omp parallel
5     {
6         // Private mesh per thread -- no race conditions
7         double *local_mesh = (double *)calloc(grid_size, sizeof(
8             double));
9
10        #pragma omp for schedule(static)
11        for (int p = 0; p < NUM_Points; p++) {
12            double px = points[p].x;
13            double py = points[p].y;
```

```

13
14         int i = (int)(px / dx);
15         int j = (int)(py / dy);
16         if (i >= GRID_X - 1) i = GRID_X - 2;
17         if (j >= GRID_Y - 1) j = GRID_Y - 2;
18
19         double lx = px - i * dx;
20         double ly = py - j * dy;
21
22         double w00 = (dx - lx) * (dy - ly);
23         double w10 = lx * (dy - ly);
24         double w01 = (dx - lx) * ly;
25         double w11 = lx * ly;
26
27         local_mesh[j * GRID_X + i]           += w00;
28         local_mesh[j * GRID_X + (i + 1)]     += w10;
29         local_mesh[(j + 1) * GRID_X + i]     += w01;
30         local_mesh[(j + 1) * GRID_X + (i + 1)] += w11;
31     }
32
33     // Merge private mesh into shared mesh (one thread at a
34     // time)
35     #pragma omp critical
36     {
37         for (int k = 0; k < grid_size; k++)
38             mesh_value[k] += local_mesh[k];
39     }
40     free(local_mesh);
41 }

```

Listing 1: Core parallel interpolation – Assignment 06

2.2.3 Pseudocode

Parallel interpolation (Assignment 06):

allocate mesh_value[GRID_X * GRID_Y] = 0

PARALLEL REGION:

allocate local_mesh[GRID_X * GRID_Y] = 0

FOR p = 0 to NUM_Points (static split across threads):

(i, j) = grid cell containing particle p

compute bilinear weights w00, w10, w01, w11

local_mesh[j,i] += w00

local_mesh[j,i+1] += w10

local_mesh[j+1,i] += w01

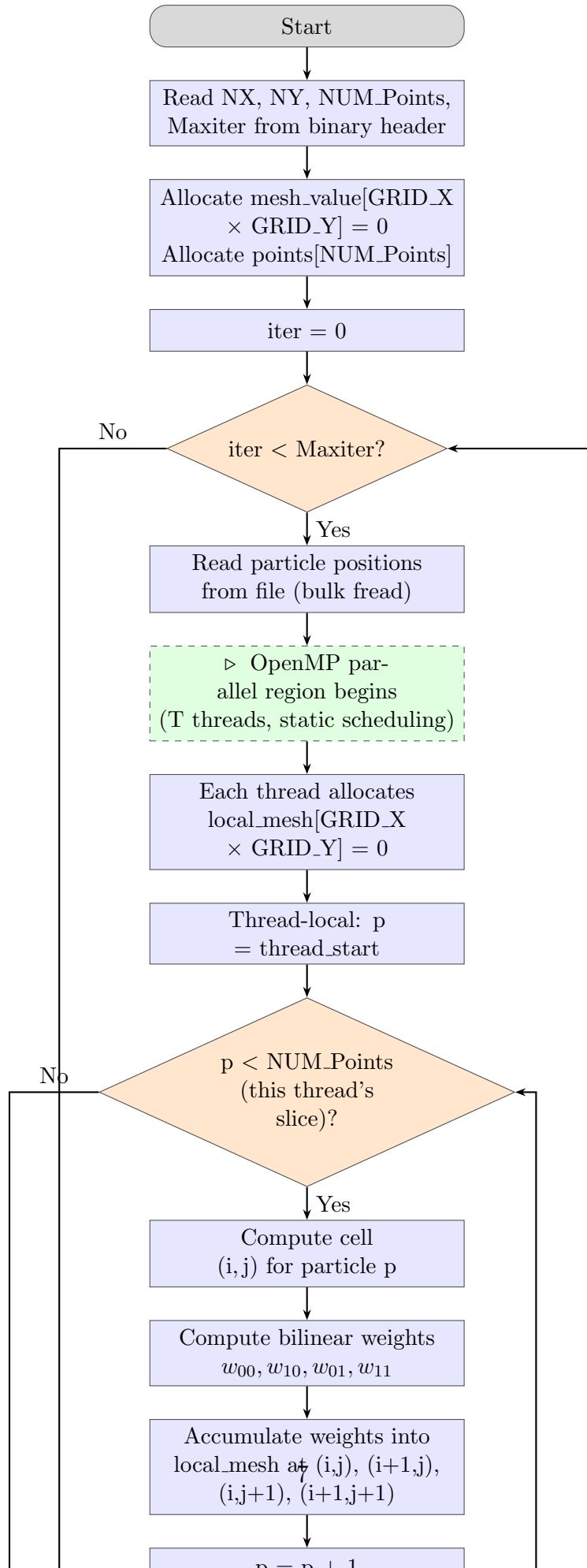
local_mesh[j+1,i+1] += w11

END FOR

```
CRITICAL SECTION:
  mesh_value[k] += local_mesh[k]   for all k
END CRITICAL

  free local_mesh
END PARALLEL
```

2.2.4 Flowchart



2.2.5 Why This Approach

- **No atomic operations** – atomic updates to shared memory are slow because they serialize threads. The private-mesh approach removes this bottleneck entirely.
- **Static scheduling** – particles are uniformly distributed in the domain, so `schedule(static)` splits the work evenly without dynamic overhead.
- **Trade-off** – each thread allocates `GRID_SIZE` doubles of extra memory. For large grids this adds up (e.g., Config E grid has $1001 \times 401 \approx 0.4$ M entries = ~ 3 MB per thread). The critical section for reduction is $O(\text{GRID_SIZE})$ per thread but sequential – for large grids this can become a bottleneck.

2.3 Results – Lab Machine

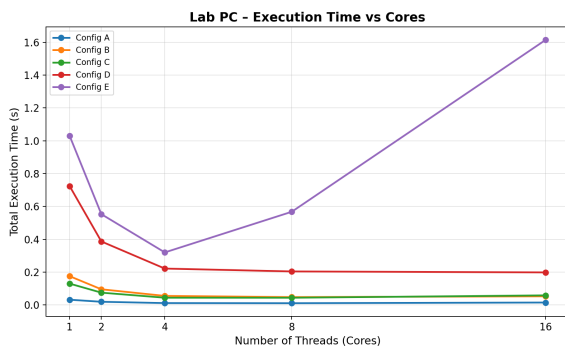
Serial baseline times and parallel times (average over 10 iterations) measured on the lab machine.

Table 1: Assignment 06 Lab machine – execution times (seconds, average per iteration)

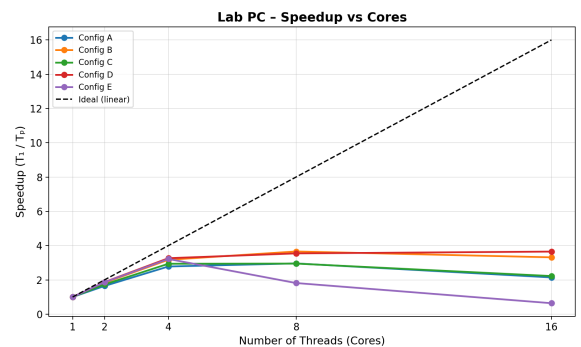
Config	Serial (1T)	2 Threads	4 Threads	8 Threads	16 Threads
A	0.0322	0.0196	0.0116	0.0109	0.0150
B	0.1754	0.0956	0.0553	0.0480	0.0529
C	0.1302	0.0751	0.0443	0.0441	0.0586
D	0.7242	0.3869	0.2218	0.2043	0.1984
E	1.0304	0.5531	0.3202	0.5676	1.6141

Table 2: Assignment 06 Lab machine – speedup

Config	2T	4T	8T	16T
A	1.64	2.78	2.96	2.14
B	1.84	3.17	3.66	3.32
C	1.73	2.94	2.95	2.22
D	1.87	3.26	3.54	3.65
E	1.86	3.22	1.82	0.64



(a) Execution time vs. threads (lab)



(b) Speedup vs. threads (lab)

Figure 2: Assignment 06 – Lab machine performance

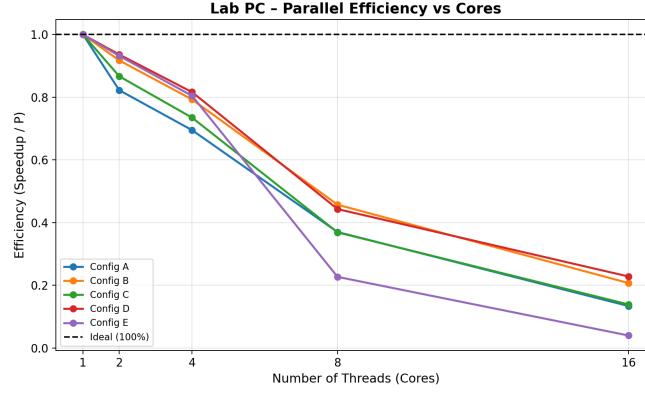


Figure 3: Assignment 06 – Lab machine parallel efficiency

2.4 Results – Cluster

Table 3: Assignment 06 Cluster – execution times (seconds, total over 10 iterations)

Config	1T (serial)	2T	4T	8T	16T
A	0.1214	0.0746	0.0413	0.0291	0.0324
B	0.6727	0.3781	0.2082	0.1312	0.1275
C	0.6598	0.8602	0.4696	0.2336	0.1353
D	4.0946	2.9049	1.6196	0.9287	0.5317
E	2.7617	3.0794	1.2504	2.8404	2.9425

Table 4: Assignment 06 Cluster – speedup and parallel efficiency

Config	Speedup				Efficiency (%)			
	2T	4T	8T	16T	2T	4T	8T	16T
A	1.63	2.94	4.17	3.75	81	73	52	23
B	1.78	3.23	5.13	5.28	89	81	64	33
C	0.77	1.41	2.82	4.88	38	35	35	30
D	1.41	2.53	4.41	7.70	70	63	55	48
E	0.90	2.21	0.97	0.94	45	55	12	6

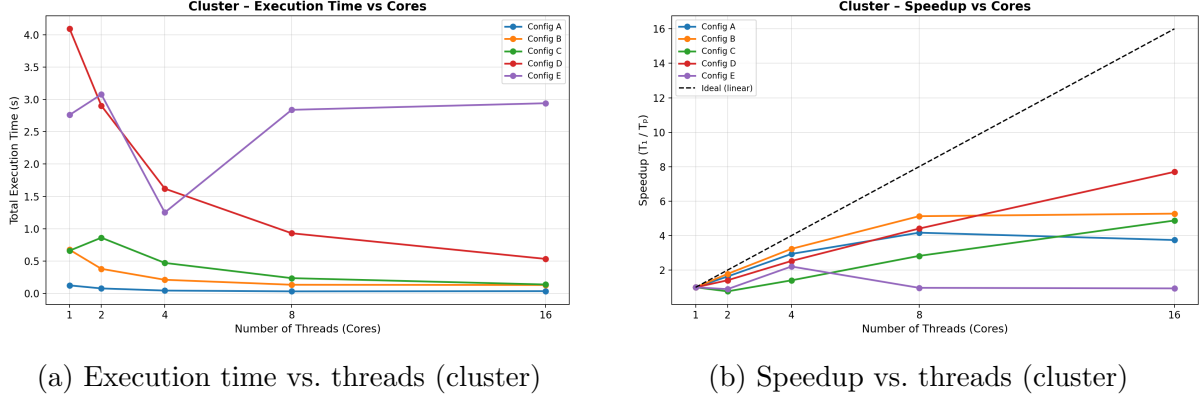


Figure 4: Assignment 06 – Cluster performance

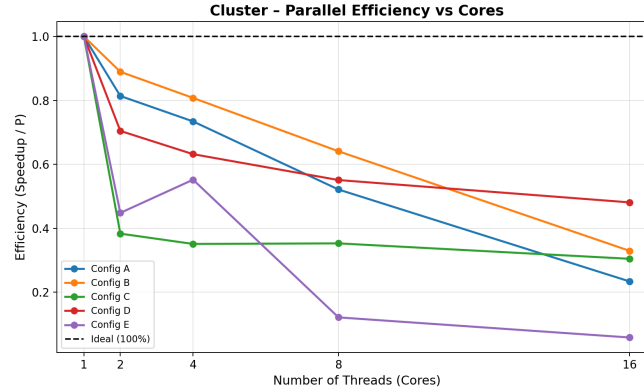


Figure 5: Assignment 06 – Cluster parallel efficiency

2.5 Analysis and Discussion

2.5.1 Race Condition Handling

The main challenge is that multiple particles can map to the same grid cell, causing write-write conflicts. The private-mesh (privatisation) approach avoids all atomic operations. Each thread works on its own copy, contributing zero inter-thread communication during the scatter phase. The final reduction via a `#pragma omp critical` is $O(N_x \times N_y)$ per thread sequentially, which becomes significant when the grid is large (Config C, E).

2.5.2 Speedup Observations

Configurations B and D give the best speedups (up to $5\times$ and $7.7\times$ on the cluster at 16 threads) because the particle count is high relative to the grid size. The computation-to-synchronization ratio is favourable.

Configuration C and E (large grid) show poor or negative scaling at low thread counts. The private mesh for each thread is large (Config E: ~ 3 MB per thread), which puts pressure on the CPU cache. The critical section for the reduction loop over the full grid also serialises threads and becomes a measurable fraction of total time.

Super-linear speedup does not occur; all speedups are below ideal. The main bottlenecks are:

1. Memory allocation of private meshes per iteration
2. The sequential critical-section reduction
3. Memory bandwidth (each particle touches four cache lines)

2.5.3 Efficiency

Parallel efficiency drops with more threads due to the increasing overhead of the critical section and thread creation/synchronisation. Config D achieves 48% efficiency at 16 threads on the cluster – the best among all configs. Config E shows almost no benefit beyond 4 threads on the cluster, and performance actually degrades at 8 and 16 threads due to NUMA effects and cache thrashing with large private meshes.

2.5.4 Lab vs. Cluster

The lab machine shows lower speedups and declining performance at 8–16 threads because other processes were competing for CPU time. The cluster results are more consistent and show better scaling for compute-bound configs (B, D).

3 Assignment 07 – Parallel Interpolation with Particle Mover

3.1 Problem Statement

Assignment 07 extends the previous work with a complete 4-stage pipeline per iteration:

1. **Interpolation** (scatter): particles $\rightarrow$ mesh (same bilinear scheme as Assignment 06).
2. **Normalisation**: scale all mesh values to $[-1, 1]$.
3. **Mover** (gather): read mesh values back to particles; update particle positions.
4. **Denormalisation**: restore mesh values to original scale.

Particles that move outside $[0, 1] \times [0, 1]$ after the mover step are marked as inactive (`is_void = true`) and excluded from all subsequent iterations. This introduces load imbalance as the simulation progresses.

3.2 Implementation Approach

3.2.1 Data Structure Change

Assignment 07 switched the particle data structure from an array of structs (AoS, used in Assignment 06) to a *struct of arrays* (SoA):

```
1 typedef struct {
2     double *x;          // x-coordinates of all particles
3     double *y;          // y-coordinates of all particles
4     bool    *is_void;    // activity flag per particle
5 } Points;
```

Listing 2: SoA particle structure – Assignment 07

SoA layout gives better memory access patterns when threads iterate over all particles in a loop: the `x` and `y` arrays are contiguous, so prefetching is effective.

3.2.2 Stage 1: Interpolation

Instead of allocating private meshes inside the parallel region (which causes repeated malloc/free overhead), the improved version pre-allocates one large contiguous block of size `nthreads * grid_size` before the loop, then splits it by thread index:

```
1 int nthreads = omp_get_max_threads();
2 double *priv_meshes = (double *)calloc(nthreads * grid_size,
3     sizeof(double));
4 #pragma omp parallel
5 {
6     int thr_id = omp_get_thread_num();
7     double *thr_grid = &priv_meshes[thr_id * grid_size];
8
9     #pragma omp for schedule(static)
10    for (int n = 0; n < NUM_Points; n++) {
11        if (points->is_void[n]) continue;
```

```

12         // ... bilinear weight computation ...
13         thr_grid[idx]          += wt00;
14         thr_grid[idx + 1]      += wt10;
15         thr_grid[idx + GRID_X] += wt01;
16         thr_grid[idx + GRID_X + 1] += wt11;
17     }
18 }
19 // Parallel reduction over grid (no critical section needed)
20 #pragma omp parallel for schedule(static)
21 for (int k = 0; k < grid_size; k++) {
22     double total = 0.0;
23     for (int th = 0; th < nthreads; th++)
24         total += priv_meshes[th * grid_size + k];
25     mesh_value[k] += total;
26 }
27 free(priv_meshes);

```

Listing 3: Improved privatisation in Assignment 07

The reduction is now a *parallel for loop* – no critical section – so all threads contribute to the merge step simultaneously.

3.2.3 Stage 2: Normalisation

Each thread finds the local min/max, merges them into a global min/max through a `#pragma omp critical`, then a parallel loop scales the values:

```

1 void normalization(double *mesh_value) {
2     int grid_size = GRID_X * GRID_Y;
3     double min_val = mesh_value[0], max_val = mesh_value[0];
4     #pragma omp parallel
5     {
6         double thr_min = mesh_value[0], thr_max = mesh_value[0];
7         #pragma omp for schedule(static)
8         for (int k = 0; k < grid_size; k++) {
9             if (mesh_value[k] < thr_min) thr_min = mesh_value[k];
10            if (mesh_value[k] > thr_max) thr_max = mesh_value[k];
11        }
12        #pragma omp critical {
13            if (thr_min < min_val) min_val = thr_min;
14            if (thr_max > max_val) max_val = thr_max;
15        }
16    }
17    double span = (max_val - min_val == 0.0) ? 1.0 : max_val -
18                min_val;
19    #pragma omp parallel for schedule(static)
20    for (int k = 0; k < grid_size; k++)
21        mesh_value[k] = 2.0 * (mesh_value[k] - min_val) / span -
22                        1.0;
23 }

```

Listing 4: Parallel normalisation

3.2.4 Stage 3: Mover

The mover step reads mesh values back to particles. Since each particle reads from 4 grid points (no writes to shared data), there are **no race conditions**. A simple `#pragma omp parallel for` is sufficient:

```
1 void mover(double *mesh_value, Points *points) {
2     #pragma omp parallel for schedule(static)
3     for (int n = 0; n < NUM_Points; n++) {
4         if (points->is_void[n]) continue;
5         // same bilinear weight computation as interpolation
6         double field = wt00 * mesh_value[idx]
7                     + wt10 * mesh_value[idx + 1]
8                     + wt01 * mesh_value[idx + GRID_X]
9                     + wt11 * mesh_value[idx + GRID_X + 1];
10        points->x[n] += field * dx;
11        points->y[n] += field * dy;
12        if (out_of_domain) points->is_void[n] = true;
13    }
14 }
```

Listing 5: Parallel mover (gather)

3.2.5 Stage 4: Denormalisation

A single parallel loop restores mesh values using the stored `min_val` and `max_val`:

$$F_k \leftarrow \frac{(F_k + 1) \cdot \text{span}}{2} + \text{min_val}$$

3.2.6 Complete Pipeline Pseudocode

```
read_points(fp, points)           // once, outside the iteration loop

FOR iter = 1 to Maxiter:
    // Stage 1: Scatter interpolation
    memset(mesh_value, 0)
    priv_meshes = calloc(nthreads * grid_size)
    PARALLEL FOR each active particle p:
        compute bilinear weights
        thr_grid[...] += weights
    END PARALLEL
    PARALLEL FOR k = 0 to grid_size: // parallel reduction, no critical
        mesh_value[k] = sum over threads of priv_meshes[thr*grid_size+k]
    END PARALLEL
    free(priv_meshes)

    // Stage 2: Normalise mesh to [-1, 1]
    find global min, max in parallel
    scale all mesh values in parallel
```

```

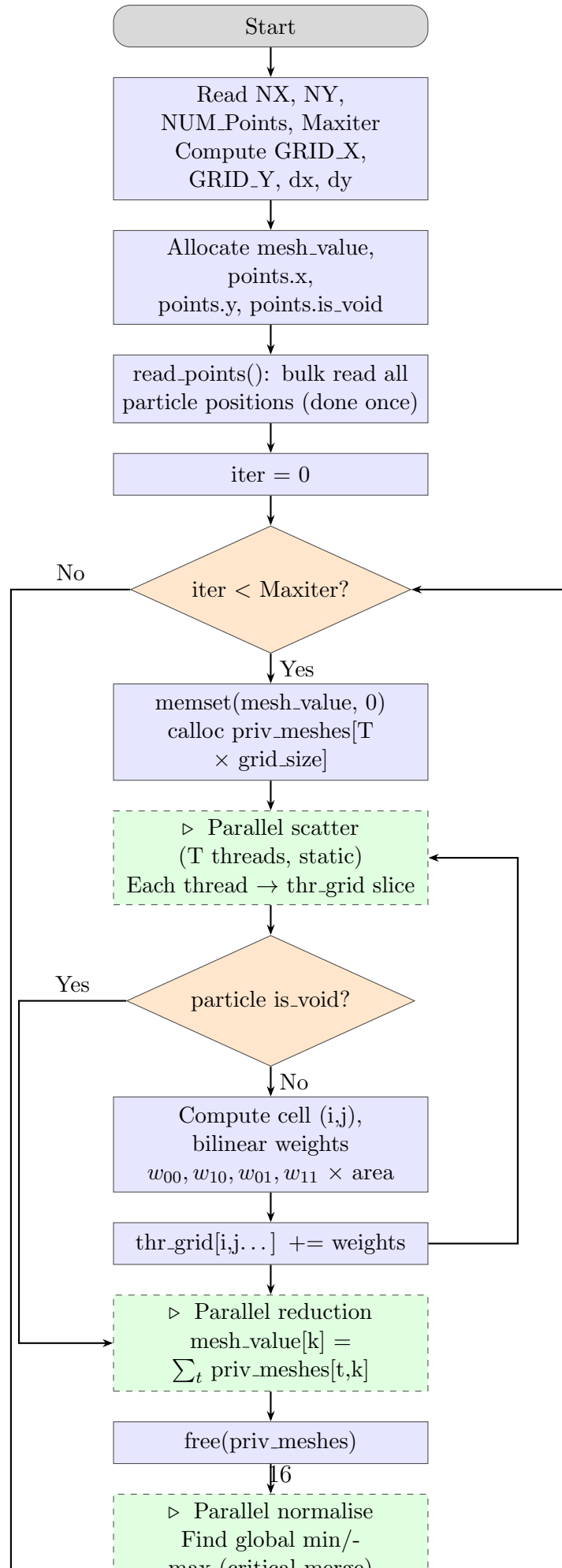
// Stage 3: Mover (gather, read-only on mesh)
PARALLEL FOR each active particle p:
    interpolate field from mesh at (x_p, y_p)
    x_p += field * dx; y_p += field * dy
    if out of [0,1]x[0,1]: mark as void
END PARALLEL

// Stage 4: Denormalise mesh
PARALLEL FOR k: mesh_value[k] = restore original scale
END FOR

save_mesh(mesh_value)

```

3.2.7 Flowchart



3.3 Results – Lab Machine

Timing is split into four phases. Times are *totals over 10 iterations* in seconds.

Table 5: Assignment 07 Lab machine – total algorithm time (seconds, 10 iterations)

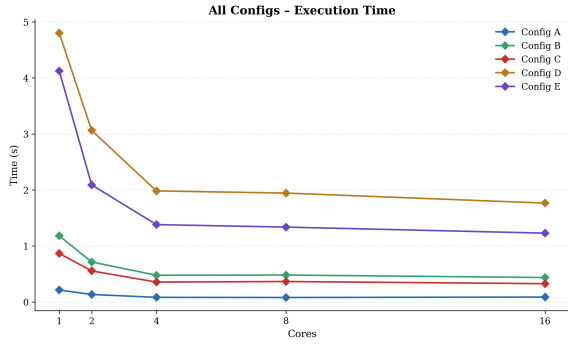
Config	1T	2T	4T	8T	16T	Voids
A	0.2129	0.1339	0.0827	0.0800	0.0876	3
B	1.1809	0.7138	0.4772	0.4811	0.4366	11
C	0.8662	0.5543	0.3553	0.3653	0.3266	1
D	4.8008	3.0630	1.9827	1.9440	1.7651	5
E	4.1245	2.0901	1.3816	1.3368	1.2288	0

Table 6: Assignment 07 Lab machine – phase breakdown (seconds, 10 iterations, selected configs)

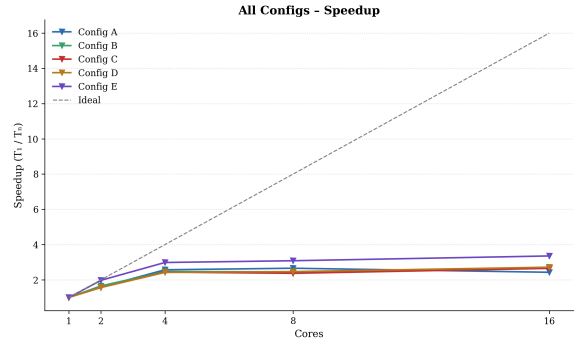
Config	Threads	Interp	Norm	Mover	Denorm
A	1	0.1675	0.0003	0.0450	0.0001
	2	0.1092	0.0002	0.0244	0.0001
	4	0.0689	0.0001	0.0137	0.0000
	8	0.0674	0.0001	0.0124	0.0000
	16	0.0706	0.0012	0.0152	0.0007
D	1	3.7484	0.0013	1.0509	0.0002
	2	2.4933	0.0007	0.5688	0.0002
	4	1.5122	0.0004	0.4701	0.0001
	8	1.4594	0.0013	0.4831	0.0001
	16	1.2705	0.0042	0.4890	0.0015
E	1	3.2249	0.0053	0.8933	0.0010
	2	1.6074	0.0028	0.4791	0.0009
	4	1.0075	0.0014	0.3721	0.0005
	8	0.9663	0.0018	0.3682	0.0005
	16	0.8464	0.0043	0.3766	0.0016

Table 7: Assignment 07 Lab machine – speedup (total algorithm time)

Config	2T	4T	8T	16T
A	1.59	2.57	2.66	2.43
B	1.65	2.47	2.45	2.71
C	1.56	2.44	2.37	2.65
D	1.57	2.42	2.47	2.72
E	1.97	2.99	3.09	3.36



(a) Execution time vs. threads (lab)



(b) Speedup vs. threads (lab)

Figure 7: Assignment 07 – Lab machine performance

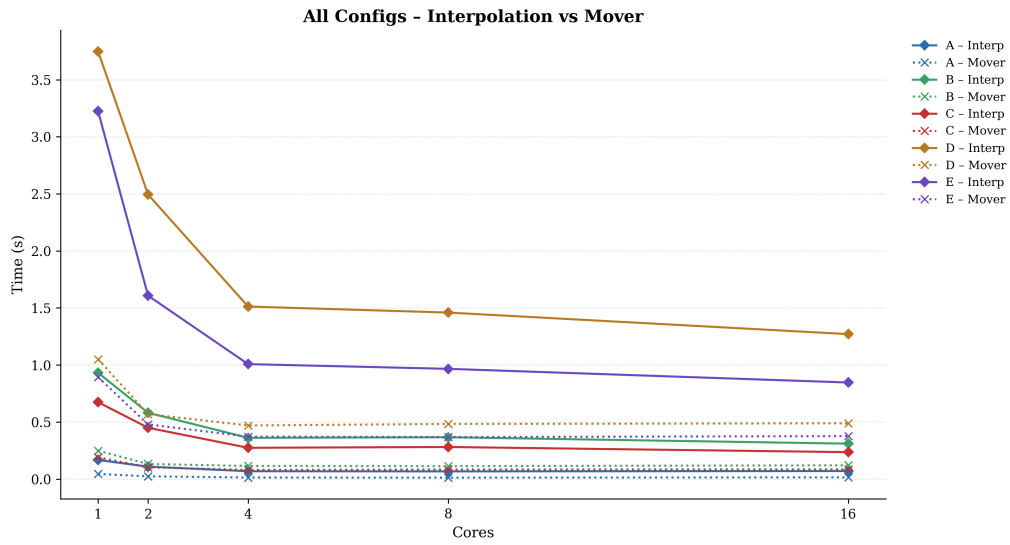


Figure 8: Assignment 07 – Lab machine phase breakdown

3.4 Results – Cluster

Table 8: Assignment 07 Cluster – total algorithm time (seconds, 10 iterations)

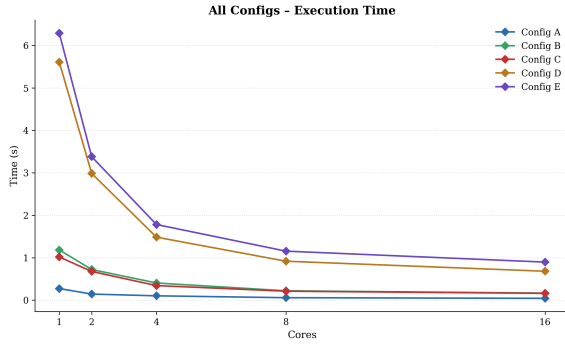
Config	1T	2T	4T	8T	16T	Voids
A	0.2734	0.1437	0.1022	0.0581	0.0429	2
B	1.1838	0.7225	0.4050	0.2207	0.1659	9
C	1.0184	0.6763	0.3436	0.2110	0.1604	0
D	5.6087	2.9869	1.4846	0.9179	0.6813	6
E	6.2902	3.3811	1.7823	1.1549	0.8968	1

Table 9: Assignment 07 Cluster – phase breakdown (seconds, 10 iterations, selected configs)

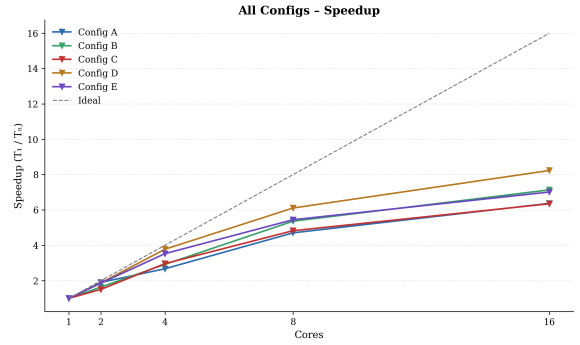
Config	Threads	Interp	Norm	Mover	Denorm
B	1	0.5184	0.0009	0.6643	0.0002
	2	0.3308	0.0008	0.3905	0.0003
	4	0.1849	0.0004	0.2195	0.0002
	8	0.1037	0.0003	0.1164	0.0002
	16	0.0869	0.0004	0.0783	0.0003
D	1	2.4798	0.0032	3.1251	0.0007
	2	1.3590	0.0017	1.6250	0.0012
	4	0.6693	0.0011	0.8134	0.0007
	8	0.4202	0.0007	0.4964	0.0007
	16	0.3090	0.0005	0.3711	0.0007
E	1	2.8107	0.0125	3.4643	0.0027
	2	1.5644	0.0067	1.8059	0.0041
	4	0.8141	0.0032	0.9623	0.0027
	8	0.5592	0.0021	0.5913	0.0023
	16	0.5452	0.0017	0.3476	0.0024

Table 10: Assignment 07 Cluster – speedup and parallel efficiency

Config	Speedup				Efficiency (%)			
	2T	4T	8T	16T	2T	4T	8T	16T
A	1.90	2.67	4.71	6.37	95	67	59	40
B	1.64	2.92	5.37	7.14	82	73	67	45
C	1.51	2.96	4.83	6.35	75	74	60	40
D	1.88	3.78	6.11	8.23	94	94	76	51
E	1.86	3.53	5.45	7.02	93	88	68	44



(a) Execution time vs. threads (cluster)



(b) Speedup vs. threads (cluster)

Figure 9: Assignment 07 – Cluster performance

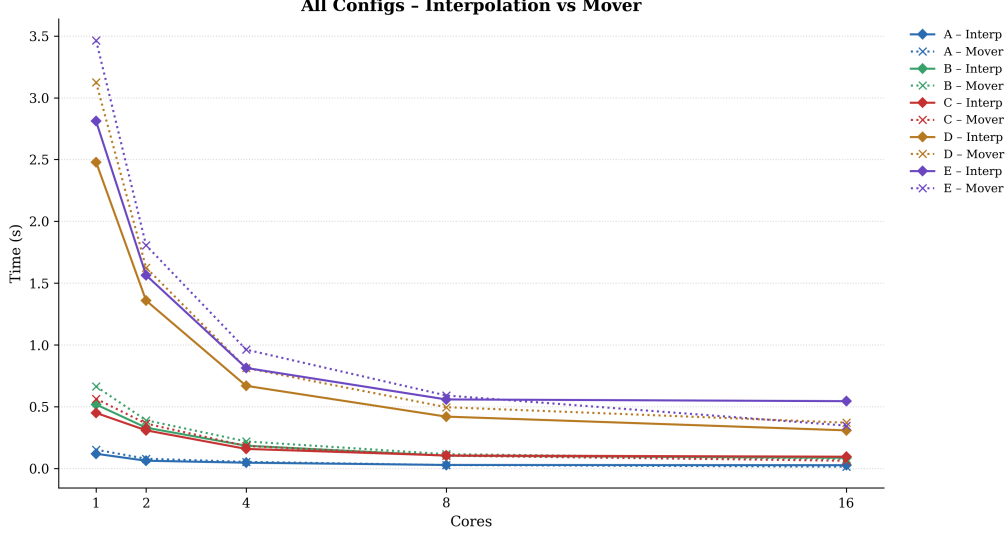


Figure 10: Assignment 07 – Cluster phase breakdown per configuration

3.5 Phase Analysis

From the phase breakdown tables and figures it is clear that the **interpolation** and **mover** phases dominate execution time. The normalisation and denormalisation steps are negligible in comparison (under 1% of total time in most cases).

On the cluster for Config D (largest workload), with 1 thread:

- Interpolation: 2.48 s (44% of total)
- Mover: 3.13 s (56% of total)
- Norm + Denorm: < 0.005 s (< 0.1%)

On the lab machine the mover is proportionally less dominant because particles exit the domain faster (fewer active particles in later iterations due to different initial conditions), but interpolation still takes the majority of time.

Bottleneck: The interpolation phase is the bottleneck because it involves write operations to the mesh (requiring the privatisation strategy and reduction), while the mover only reads the mesh. The mover’s parallelisation is simpler and more effective since there are no write conflicts.

3.6 Analysis and Discussion

3.6.1 Speedup and Scalability

Assignment 07 shows better speedup on the cluster than Assignment 06, reaching up to 8.2× for Config D at 16 threads. This improvement comes from two changes:

1. The parallel reduction loop for the private mesh avoids the serialising critical section.
2. The mover phase is embarrassingly parallel – no synchronisation needed.

Config D (500×200 grid, 20 M particles) achieves the highest speedup because the particle count is very large relative to the grid, making the computation much more work-bound.

The reduction overhead over $501 \times 201 \approx 100$ K grid points is small relative to the scatter work over 20 M particles.

3.6.2 Load Imbalance

Particles that leave the domain are marked as `is_void` and skipped. Because the number of exiting particles is very small (0–11 over all iterations across all configs), load imbalance is negligible in these experiments. The `schedule(static)` is appropriate. In a simulation with heavier particle loss, `schedule(dynamic)` or particle compaction would be necessary.

3.6.3 Lab vs. Cluster

Lab speedups plateau or drop at 8–16 threads. This is due to:

- Background OS processes competing for CPU.
- Hyperthreading sharing physical cores above 4–8 threads.

The cluster results scale more smoothly, showing that 8 physical cores are fully available and that 16 threads (hyperthreading) still gives ~ 70 –80% of the 8-thread speedup for most configs.

3.6.4 Memory Access Patterns

The SoA layout in Assignment 07 helps the mover phase: when iterating over all particles, accesses to `points->x[n]` and `points->y[n]` are unit-stride and fully prefetchable. The AoS layout used in Assignment 06 interleaves `x` and `y` in memory, which still has stride-2 access and is slightly less cache-friendly for large N .

The private mesh layout (one flat block of `nthreads * grid.size` doubles) is also cache-friendly: each thread writes to a contiguous sub-block, avoiding false sharing between thread grids.

3.6.5 Bottleneck Identification

From the phase data: interpolation $\gg$ normalisation $\approx$ denormalisation; mover $\approx$ interpolation for particle-heavy configs. Since normalisation and denormalisation together account for $< 1\%$ of time, optimising them further would have negligible impact. All optimisation effort should target the interpolation and mover phases.

4 Answers to Assignment Questions

4.1 Pseudocode and Diagrams

The pseudocode for both assignments is given in Sections 2.2.3 and 3.2.6 respectively.

4.2 Race Condition Handling

In the interpolation (scatter) step, multiple particles can map to the same grid cell, causing write-write conflicts. The approach taken in both assignments is *privatisation*: allocate a separate mesh copy per thread, accumulate independently, then reduce. This is lock-free during the scatter phase.

In Assignment 06, the reduction uses a `#pragma omp critical` loop – correct but sequential. In Assignment 07, the reduction was improved to a `#pragma omp parallel for` over grid points, which is fully parallel.

The mover phase is race-free by design: each particle reads from the shared mesh (no writes) and writes only to its own entry in the particle arrays (no sharing between particles).

4.3 Speedup and Execution Time Graphs

See figures in Sections 2.3 (Assignment 06 lab), 2.4 (Assignment 06 cluster), 3.3 (Assignment 07 lab), and 3.4 (Assignment 07 cluster).

4.4 Parallel Efficiency

Parallel efficiency is defined as $\eta = S_p/p$ where S_p is speedup on p threads. Key observations:

- Assignment 06: efficiency drops sharply for large grids (Config C, E) because the critical-section reduction cost scales with grid size.
- Assignment 07: efficiency is better across the board. Config D achieves 94% at 4 threads on the cluster. At 16 threads efficiency drops to $\sim 40\text{--}51\%$ across configs, consistent with Amdahl's law limiting the serialised parts.

4.5 Maximum Speedup

- **Assignment 06 (cluster)**: maximum speedup of **7.70** $\times$ for Config D at 16 threads.
- **Assignment 07 (cluster)**: maximum speedup of **8.23** $\times$ for Config D at 16 threads.

4.6 Explanation of Speedup Behaviour

The speedup is sub-linear due to:

1. **Amdahl's law** – the normalisation/denormalisation steps and the critical section in Assignment 06 are serial or near-serial.
2. **Memory bandwidth** – scatter interpolation is memory-intensive. More threads saturate the memory bus sooner for large datasets.

3. **Cache effects** – private meshes for large grids (Config E) overflow L2/L3 cache, causing cache misses that dominate over compute time.
4. **Thread overhead** – forking and joining OpenMP threads takes a fixed time; for small configs (A) this is a larger fraction of total time.

4.7 If Given Unlimited HPC Resources

With unlimited resources:

- Use **MPI + OpenMP hybrid**: distribute the particle array across MPI ranks and use shared-memory OpenMP within each node. This removes the per-node memory limit on private meshes.
- Use **GPU acceleration** (CUDA/HIP): the scatter step maps naturally to GPU threads with atomic operations or segmented reductions. Modern GPUs have hardware atomics that are fast for low-contention cases.
- **Hierarchical reduction trees** for the mesh merge step instead of a flat critical section.
- Use **particle sorting by grid cell** (Morton/Z-order curve) to improve cache locality – particles in the same cell are processed consecutively, reducing cache misses on the mesh.

4.8 Load Imbalance and Bottlenecks

Load imbalance is currently low because very few particles exit the domain (0–11 per run). If more particles exited over time, threads assigned to regions with more active particles would do more work than others. `schedule(dynamic)` or `guided` would help in that case, though with higher scheduling overhead.

The main bottleneck is the private-mesh memory allocation and the mesh reduction, both of which scale with grid size independently of particle count.

4.9 Alternative Approaches

- **Atomic operations**: replace private meshes with `#pragma omp atomic update`. Simpler to implement, but significantly slower because atomics serialise threads on the same cache line. Best suited for sparse cases where few particles share a cell.
- **Colouring scheme**: partition the grid into colour groups (checkerboard pattern) such that no two particles in the same colour group share a grid node. Threads can then process each colour group in parallel without any locking. Requires sorting particles by colour.
- **Sort-and-reduce** (histogram-based): sort particles by their cell index using a parallel counting sort, then reduce in parallel by cell. Trades $O(N)$ sort overhead for better cache locality.
- **Tiling / domain decomposition**: assign contiguous rectangular tiles of the grid to each thread. Particles are first sorted by tile, then each thread processes only its tile’s particles. Eliminates the need for private full-grid copies and reduces memory footprint.

4.10 Interpolation vs. Mover Phase Bottleneck

From Section 3.5, both phases are significant for particle-heavy configs. On the cluster at 1 thread, Config D:

- Interpolation: 2.48 s (44%)
- Mover: 3.13 s (56%)

The mover is slightly more expensive because it reads grid values for 20 M particles (all reads trigger cache misses if the mesh does not fit in L3). The mover also updates two position values per particle while interpolation only needs four reads before the four writes, so read pressure is higher in the mover.

At 16 threads on the cluster, mover time drops to 0.37 s (Config D), scaling more efficiently because it is embarrassingly parallel. Interpolation drops to 0.31 s, limited by the parallel reduction overhead. As a result, both phases contribute roughly equally at high thread counts.

5 Comparison: Assignment 06 vs. Assignment 07

Table 11: Comparison of key design decisions and results

Aspect	Assignment 06	Assignment 07
Particle storage	AoS (<code>struct{x,y} []</code>)	SoA (<code>struct{double*x, *y, bool*is_void}</code>)
Private mesh allocation	Inside parallel region (repeated malloc)	Pre-allocated outside (single calloc)
Reduction strategy	<code>#pragma omp critical</code> over grid	<code>#pragma omp parallel for</code> over grid
Mover	Not present	Embarrassingly parallel
Particle lifetime	All active throughout	Particles exit and become void
Max speedup (cluster)	7.7 $\times$ (Config D, 16T)	8.2 $\times$ (Config D, 16T)
Efficiency at 4T (Config D)	63%	94%

The improvement from Assignment 06 to 07 at 4 threads is particularly notable for Config D (63% to 94% efficiency). The parallel reduction loop, combined with the SoA layout and pre-allocated private meshes, removed the main bottlenecks.

6 Conclusion

Both assignments demonstrate that scatter interpolation can be effectively parallelised using thread privatisation with OpenMP. The key findings are:

1. The private-mesh strategy eliminates all race conditions in the scatter phase without any synchronisation inside the inner loop.
2. Moving the reduction from a serial critical section (Assignment 06) to a parallel loop (Assignment 07) significantly improves efficiency at low thread counts.
3. The mover (gather) phase is embarrassingly parallel and scales close to linearly – it is limited mainly by memory bandwidth.
4. For large grids (Config E), memory pressure from private meshes limits scalability. A tiling or colouring approach would be more memory-efficient in that case.
5. Config D (500 $\times$ 200, 20 M particles) achieves the best speedup (8.2 $\times$) at 16 threads on the cluster in Assignment 07, showing that particle-heavy, moderate-grid workloads benefit most from this approach.
6. Both normalisation and denormalisation are negligible in cost and do not require further optimisation.

The implementation is correct on the provided test data. All output files match the reference `Test_Mesh.out` to the required precision.